Extração e Preparação de Dados

Aula 10 – Feature Engineering e Manipulação de datas

ibmec

Quem sou eu?

Professor: Luís Aramis dos Reis Pinheiro.

- **Doutorado e Mestrado** em **Ciências Mecânicas** – **UnB** – **CAPES 7**
- **Graduação** em Licenciatura em Física – **UNIFAP**

luis.pinheiro@professores.ibemec.edu.br

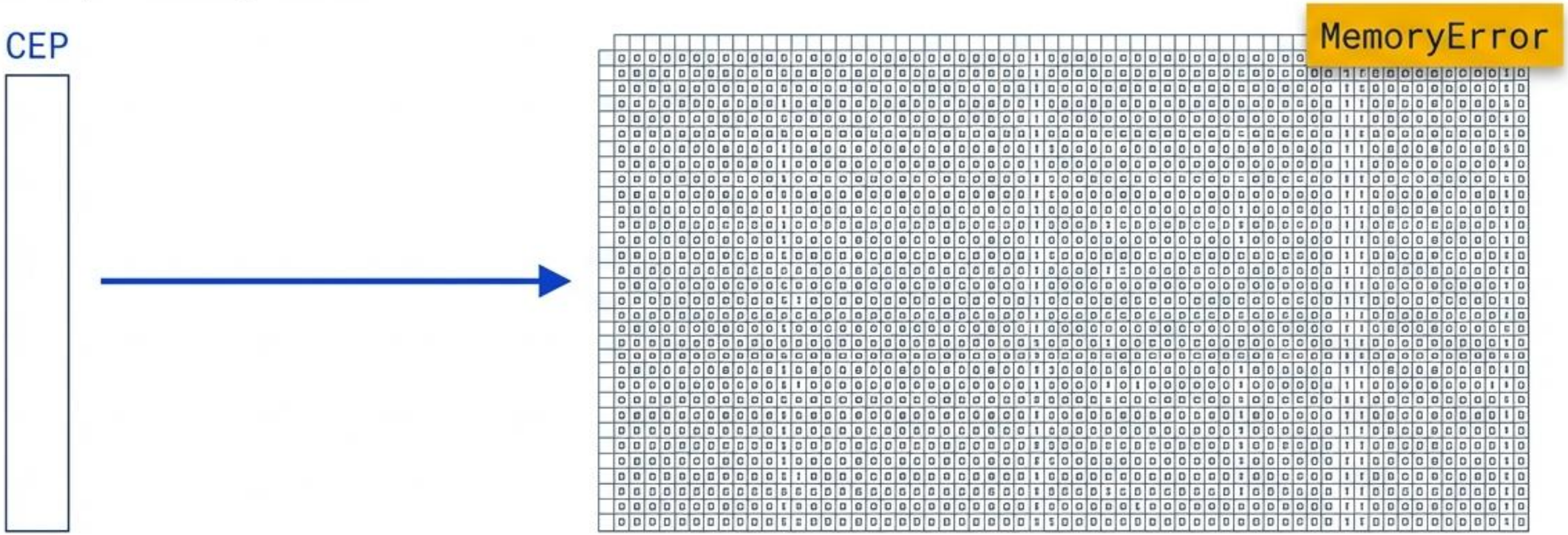
(96) 99907-5819

@l_aramis

Aula passada

O Paradigma do CEP: O Limite da Matriz

- **A Barreira do `One-Hot Encoding`:** Inviável para variáveis textuais com altíssima cardinalidade.
- **Explosão de Dimensionalidade:** Uma única coluna de `CEPs` transforma-se em milhares de novas colunas.
- **Ameaça Computacional:** Geração imediata de `MemoryError` e criação de matrizes esparsas inúteis para o algoritmo.



O CEP COMO VARIÁVEL CHAVE NA DETECÇÃO DE FRAUDE (E-COMMERCE & CARTÃO DE CRÉDITO)

COMO O MODELO ANALISA O CEP (TARGET ENCODING)

A TAXA HISTÓRICA DE FRAUDE DO CEP É USADA COMO PONTUAÇÃO (TARGET ENCODING)

1. PADRÕES DE QUADRILHAS E ROTAS

50+ CARTÕES DIFERENTES

PICO DE COMPRAS COM CARTÕES CLONADOS PARA O MESMO ENDEREÇO DE ENTREGA

TAXA HISTÓRICA DISPARA

2. ENDEREÇOS DE ALTO RISCO (GALPÕES FANTASMAS)

LOCAIS USADOS PRIMARIAMENTE PARA RECEBER MERCADORIAS COMPRADAS COM FRAUDE

ENDEREÇOS NÃO VERIFICADOS OU NÃO EXISTENTES

3. ROUBO DE CARGA E PERDA LOGÍSTICA

18% TAXA DE ROUBO DE CARGA

CEPs COM ALTAS TAXAS DE ROUBO DE CARGA NO TRAJETO

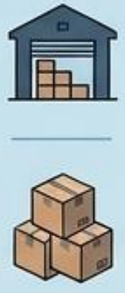
CLASSIFICAÇÃO DO SISTEMA: FRAUDE/PERDA LOGÍSTICA

COMO FREQUENCY E TARGET ENCODING RESOLVEM O PROBLEMA DO CEP EM DETECÇÃO DE FRAUDE

RESPOSTA AO CENÁRIO DE QUADRILHAS, GALPÕES E ROUBOS (DE <IMAGE 0>)

1. ENTRADA DE DADOS BRUTOS (TRANSAÇÕES)

Transação ID	CEP	Número de cartas	Classe de Fraude
1	01001-001	0	1 = Fraude
⋮			0 = Legitimo
	22750-020	0	0 = Legitimo
	01001-001	1	1
	01001-001	1	1
	01001-001	1	1
	01001-001	1	1
	01001-001	1	1
	01001-001	1	1
	01001-001	1	1
	01001-001	1	1
	22750-020	0	



LOCAIS USADOS PRIMARIAMENTE PARA RECEBER MERCADORIAS COMPRADAS COM FRAUDE

2. PROCESSAMENTO DE CODIFICAÇÃO (FEATURE ENGINEERING)

CODIFICAÇÃO DE FREQUÊNCIA (FREQUENCY ENCODING)

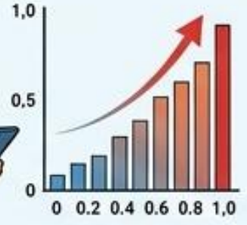
CEP	TRANTAÇÕES
01001-001	500
22750-020	50
01001-001	50
22750-020	10
22750-020	5

= CONTAGEM DE TRANSAÇÕES POR CEP

VOLUME DE TRANSAÇÕES

AUMENTO DE VOLUME SINALIZA ATIVIDADE ANORMAL

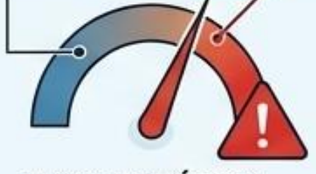
CODIFICAÇÃO DE TARGET (TARGET ENCODING)



CEP	98%

= MÉDIA (FRAUDE CLASS) POR CEP

FRAUDE DE FRASE → TAXA DELIGRO



TAXA HISTÓRICA DE FRAUDE

TAXA HISTÓRICA DISPARA PARA CEPs FRAUDULENTOS

3. SAÍDA DE DADOS PARA O MODELO

CEP_Frequency_Encoded	CEP_Target_Encoded
01001-001	98%
22750-020	
22750-020	
01001-001	

MODELO DE DETECÇÃO DE FRAUDE: RISCO ALTO IDENTIFICADO PARA CEP 01001-001

FRAUDE DETECTADA

PREVENÇÃO DE QUADRILHAS

BLOQUEIO DE ENDEREÇOS DE RISCO

Frequency Encoding: O Peso da Categoria

- **A Lógica:** Substitui a categoria (string) pela sua frequência relativa (peso) no dataset.
- **Eficiência:** Mantém a estrutura de uma única coluna (1D), eliminando a explosão de matrizes.
- **Implementação Pandas:** Mapeamento direto e veloz utilizando os métodos `.value_counts(normalize=True)` e `.map()`.

Cidade:
Rio de Janeiro → **Peso Operacional: 0.15 (15%)**

Target Encoding: A Probabilidade Histórica

- **O Conceito Preditivo:** Substitui a categoria pela média histórica da variável alvo (Target).
- **Mapeamento de Risco:** Traduz o nome do bairro ou CEP pela sua probabilidade matemática.
- **Atenção (Data Leakage):** Exige rigor estatístico; deve ser calculado estritamente nos dados de treino para evitar vazamento do futuro.

CEP (String) ⟶ `GroupBy(CEP)['Inadimplência'].mean()` ⟶ **Risco: 0.82**

Check de Aprendizado: cut vs qcut

- Avaliação estratégica do cenário de negócios para escolha da função.

Situação A

Exige regras estáticas e cortes matemáticos inegociáveis.

Exemplo: Notas 0-10 -> Reprovado/Aprovado

Resposta: pd.cut

Situação B

Exige distribuição populacional rigorosamente igualitária.

Exemplo: Classes de Poder Aquisitivo A/B/C com o mesmo volume de clientes

Resposta: pd.qcut

A Função pd.cut: Estruturação de Intervalos Absolutos

1. `bins` **como um Número Inteiro (** `int` **)**

Python

```python
import pandas as pd
idades = pd.Series([15, 20, 22, 45, 60, 78])

# Divide em 3 grupos de larguras iguais (ex: 15-36, 36-57, 57-78)
grupos = pd.cut(idades, bins=3)
```

Quando usar: Quando você quer que a *distância numérica* de cada grupo seja a mesma, independentemente de quantos itens caiam dentro de cada grupo.

2. `bins` como uma Sequência de Escalares (Lista, Tupla ou Array)

Python

```python
idades = pd.Series([15, 20, 22, 45, 60, 78])

# Limites exatos: 0 a 18 (menor), 18 a 60 (adulto), 60 a 100 (idoso)
limites = [0, 18, 60, 100]
grupos = pd.cut(idades, bins=limites, labels=["Menor", "Adulto", "Idoso"])
```

Quando usar: Quando você tem uma **regra de negócio específica** ou categorias pré-definidas. Por exemplo, faixas etárias do IBGE, faixas de imposto de renda, ou limiares de risco.

3. `bins` como um `IntervalIndex`

Python

```python
# Criando intervalos manualmente
meus_intervalos = pd.IntervalIndex.from_tuples([(0, 20), (20, 40), (40, 80)])

grupos = pd.cut(idades, bins=meus_intervalos)
```

Quando usar: Quando você já gerou esses intervalos em uma análise anterior (usando outro `pd.cut` e extraindo os `.cat.categories`) e quer aplicar **exatamente os mesmos intervalos** em um novo conjunto de dados (como dados de teste ou dados de um mês diferente) para garantir consistência perfeita.

Feature Engineering e Manipulação de Datas

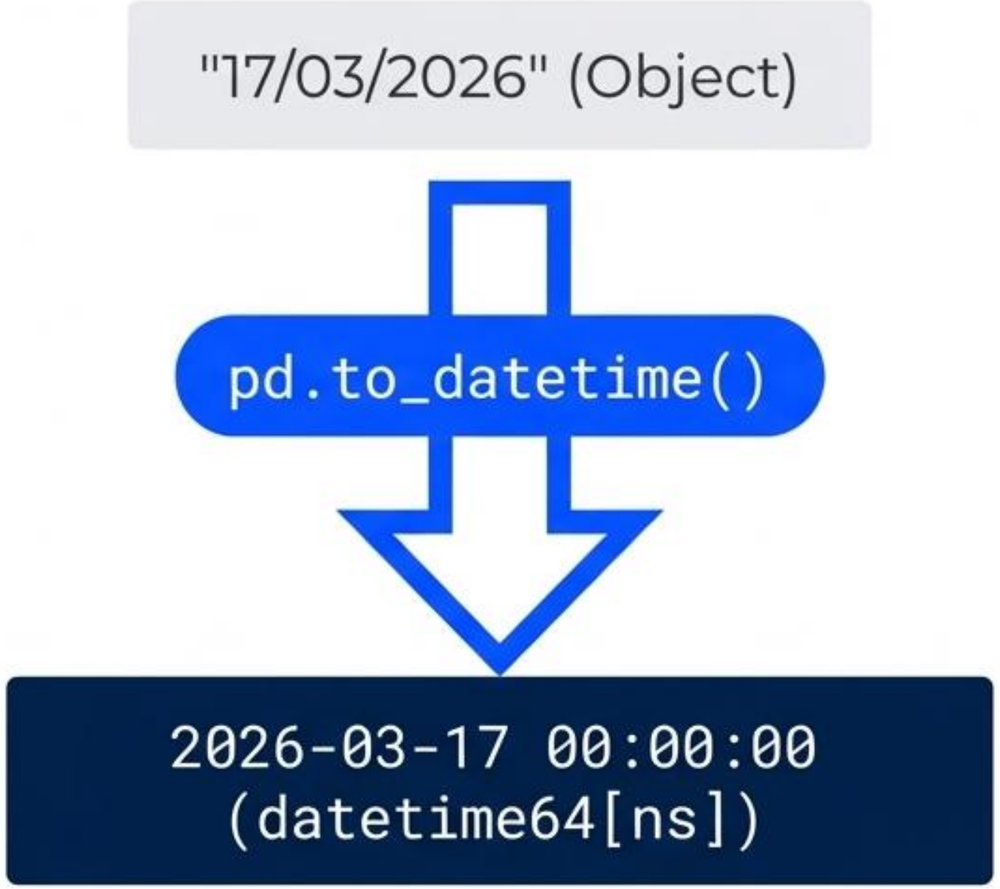
- Extraindo Sinais Analíticos do Tempo
- Disciplina: Extração e Preparação de Dados (IBM8915)
- Professor: Luís Aramis
- Data: 17/03/2026

O Paradigma do Tempo e a Cegueira Sazonal

- **A Ilusão do Texto:** "17/03/2026" é estatisticamente inútil como string.
- **Cegueira Sazonal:** Modelos não deduzem feriados ou fins de mês sozinhos.
- **A Missão:** Decompor datas unificadas em dezenas de colunas preditivas.

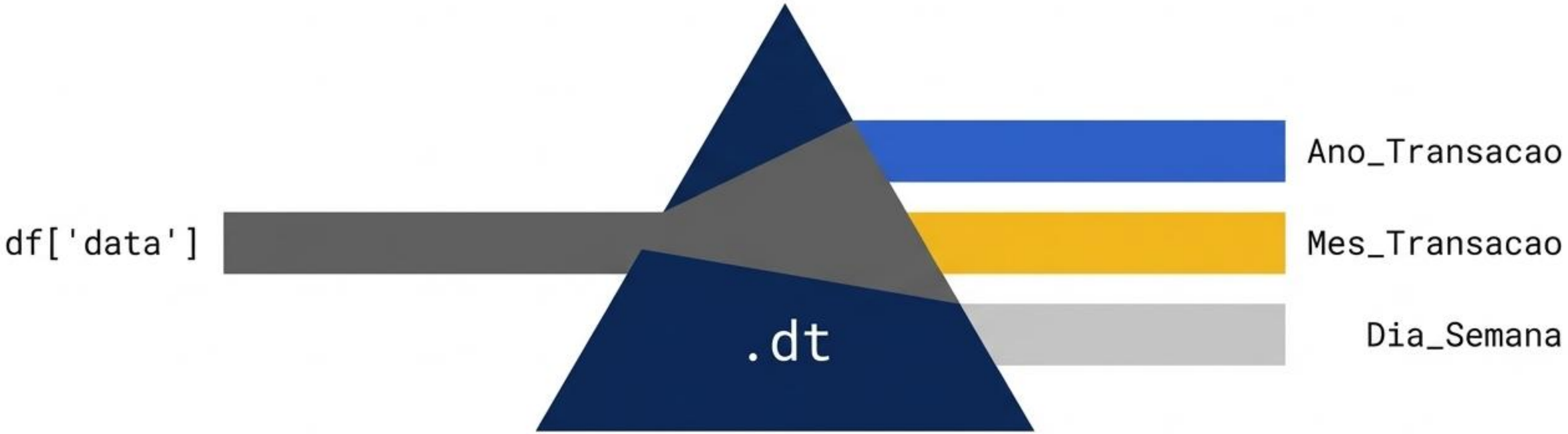
Conversão Temporal no Pandas

- O formato Object (String) bloqueia operações temporais.
- O método `pd.to_datetime()` força a tradução para a máquina.
- Padronização nativa: Ano-Mês-Dia Hora:Minuto:Segundo.
- Parâmetro crítico: `dayfirst=True` para o padrão brasileiro.



Dissecação com o Acessor dt

- O acessor `.dt` atua como um "bisturi" analítico.
- Extração direta de características: `.dt.year`, `.dt.month`, `.dt.day`.
- Captura de sazonalidade: `.dt.dayofweek` (0=Segunda, 6=Domingo).



Check de Aprendizado (A Lógica do Fim de Semana)

- **Cenário**: O marketing exige uma coluna booleana (`Is_Weekend`).
- **Dado atual**: Temos o dia da semana via `.dt.dayofweek` (0 a 6).
- **Pergunta**: Qual operação lógica simples e vetorizada cria essa flag preditiva de uma só vez?

Check de Aprendizado
(A Lógica do Fim de Semana)

```
df['Is_Weekend'] = (df['Data_Venda'].dt.dayofweek >= 5).astype(int).
```

Os dias 5 e 6 são sábado e domingo. Tudo que for maior ou igual a 5 vira True (1), o resto vira False (0).

Vocês acabaram de inventar uma *feature* nova e valiosíssima para qualquer modelo de crédito ou varejo!"

Matemática Temporal (Deltas)

- Subtração de datas gera um objeto especial: `timedelta`.
- Recência importa mais que datas estáticas (ex: Idade da Conta).
- A Equação Base: `Data Atual - Data Histórica = Delta`.
- O bisturi numérico: extração dos dias brutos com `.dt.days`.

[Data da Transação] − [Data de Registro] = [Tempo de Resposta]

(timedelta)

↓ `.dt.days`

45 dias

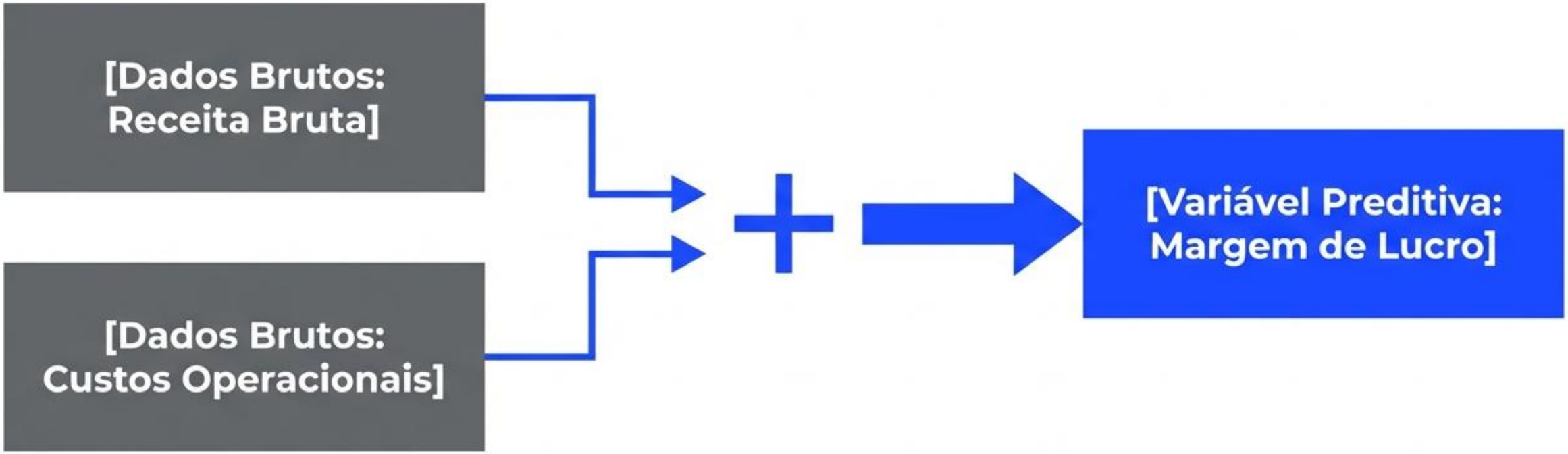
Hands-on: Engenharia do Tempo (Lab 11)

`lab_11_datetime.ipynb`

- 1. Conversão: Aplicar to_datetime nas datas problemáticas.
- 2. Dissecação: Criar as colunas Ano_Transacao e Dia_Semana.
- 3. A Flag: Implementar a lógica para criar a variável Is_Weekend.
- 4. Delta: Calcular os dias entre o início do contrato e a transação.

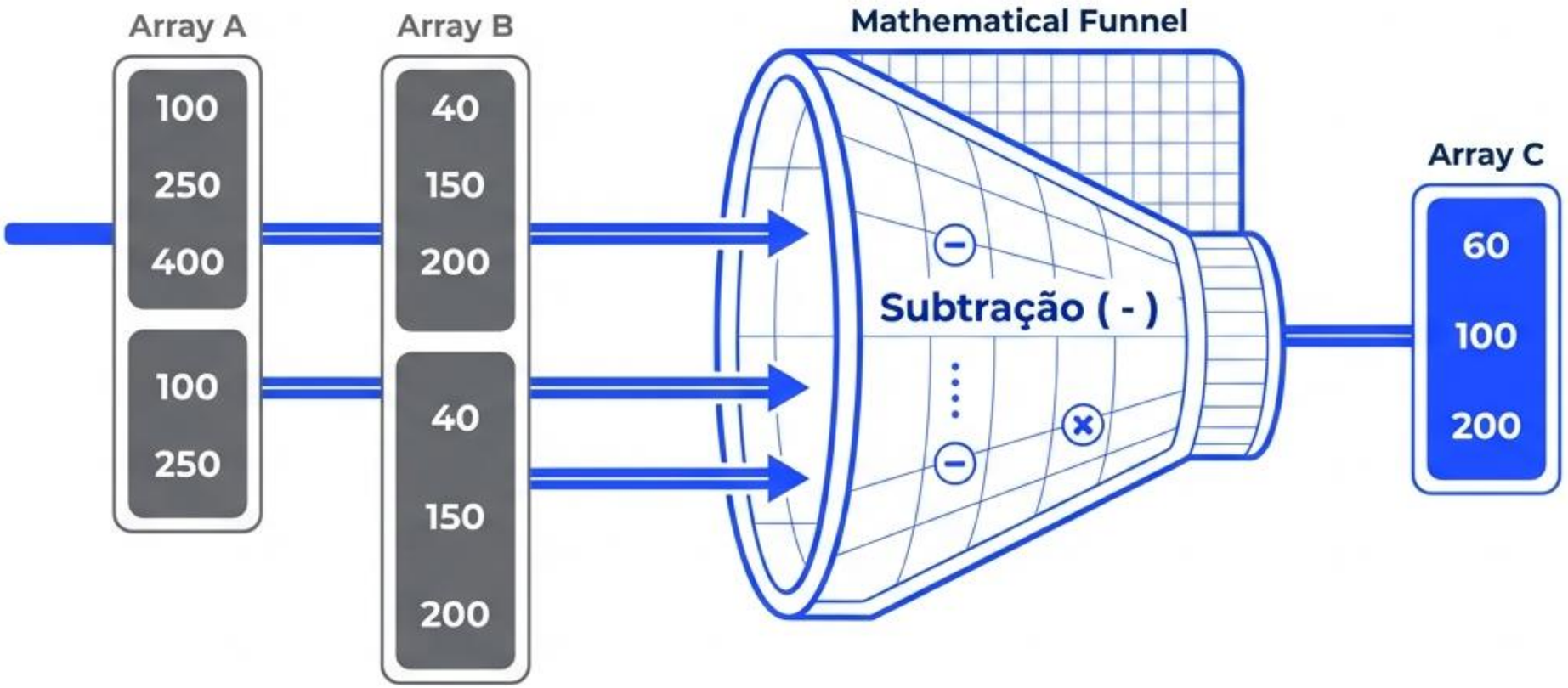
Engenharia de Atributos Numéricos

— A inteligência analítica reside na proporção matemática, não no valor absoluto isolado.
— Modelos preditivos dependem da entrega de relações lógicas pré-calculadas.
— A conversão de variáveis independentes em indicadores diretos de negócio.



Aritmética de Séries e Vetorização

- O Pandas executa operações matemáticas alinhando os índices automaticamente.
- Eliminação de loops iterativos: o cálculo ocorre de matriz contra matriz simultaneamente.
- Otimização de processamento em larga escala executada na camada C subjacente.



Sem for loops.
Cálculo paralelo.

O Paradigma do Código Poluído

- A sintaxe tradicional de atribuição fragmenta a lógica de construção.
- Criação excessiva de estados mutáveis e DataFrames temporários na memória.
- Quebra arquitetural do Pipeline: inviabiliza a automação via orquestradores.

Anti-Pattern

```
df['receita_bruta'] = df['vendas'] * df['preco']
df['custo_total'] = df['impostos'] + df['frete']
df['lucro'] = df['receita_bruta'] - df['custo_total']
```

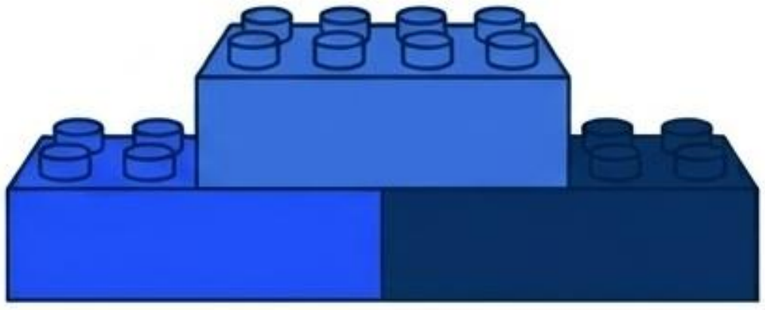
Broken Pipeline

Fluxo Interrompido (Mutável)

Orquestração Limpa: O Método .assign()

- Declaração de múltiplas variáveis simultâneas em uma única chamada de método.
- Preservação da imutabilidade e garantia de um fluxo de dados direcional.
- Preparação direta da arquitetura de código para orquestradores (Apache Airflow).

```
df_final = df.assign(
    receita_bruta = lambda x: x.vendas * x.preco,
    custo_total = lambda x: x.impostos + x.frete,
    lucro = lambda x: x.receita_bruta - x.custo_total
)
```



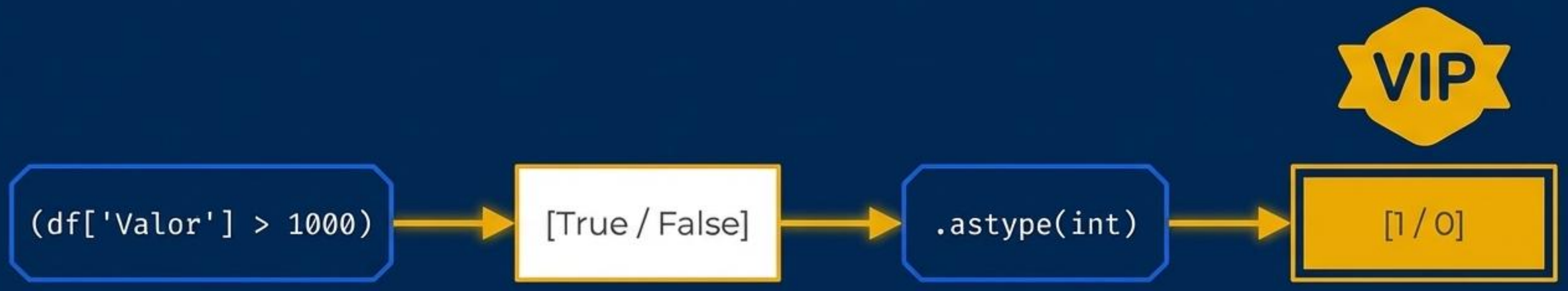
Method Chaining (Imutável)

Não é Padrão

```
# 1. Cria-se uma lista vazia para guardar os resultados
lista_lucros = []

# 2. Inicia-se o laço FOR iterando pelo número total de linhas
for i in range(len(df)):

    # 3. O Python acessa a linha 'i' de Receita e a linha 'i' de Custo e subtrai
    lucro_da_linha = df['Receita'][i] - df['Custo'][i]

    # 4. Adiciona o resultado na lista
    lista_lucros.append(lucro_da_linha)

# 5. Finalmente, anexa a lista preenchida como uma nova coluna no DataFrame
df['Lucro'] = lista_lucros
```

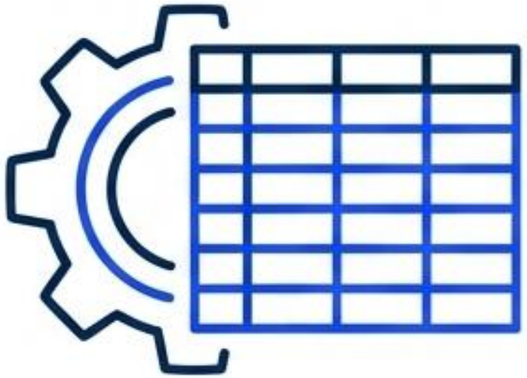
Check de Aprendizado: A Flag de Negócio

- **Regra de Negócio:** Clientes VIPs possuem compras estritamente maiores que R$ 1.000,00.
- **Desafio:** Modelos estatísticos compreendem apenas matrizes numéricas binárias (1 e 0).
- **Solução:** Aplicação de casting lógico para tradução preditiva nativa.



Hands-on: Engenheiros Financeiros (Lab 08)

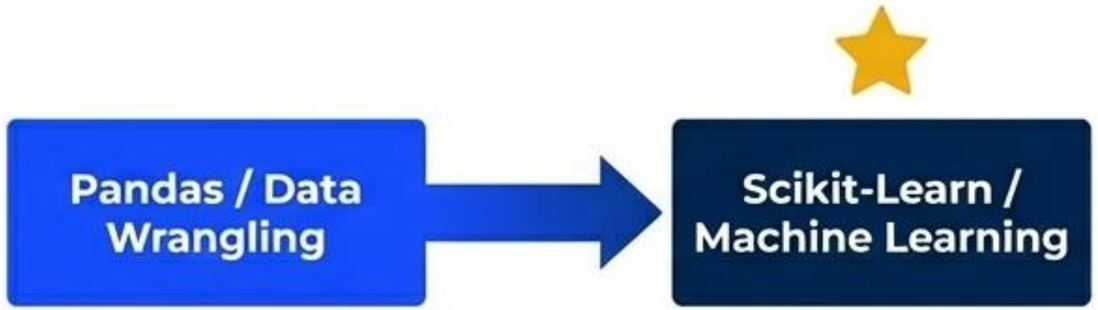
- **Carga e Limpeza**: Ingestão do CSV e encadeamento de `".dropna()"` nas variáveis monetárias.
- **Novas Métricas (Subtração):** Geração da feature Lucro Total utilizando orquestração com `".assign()"`.
- **Proporções (Divisão):** Criação da feature Ticket Médio cruzando a receita com o volume de itens.
- **Lógica Booleana:** Inserção da flag Is_VIP integrada nativamente ao encadeamento de código.



Versionamento e o Fim da Preparação Clássica

- **Fechamento do Portfólio:** Entrega dupla dos laboratórios focados na transformação de atributos.
- **Maturidade do Dado:** A base agora possui uma estrutura numérica inteligente, limpa e orquestrada.
- **Próxima Fase (Aula 12):** Transição para o ecossistema Scikit-Learn e escalonamento matemático.

```
$ git add .
$ git commit -m 'feat: feature engineering'
$ git push
```



Dúvidas?

CRIATIVIDADE
INOVAÇÃO
¡DEIA!
COMAMUNIDADE
CRIATIVIDADE
BRAINSTORM
INOVAÇÃO

ibmec

A direção do amanhã.

/ibmec